

Geometric Foundations of Data Analysis I

Joshua Maglione

September 11, 2026

Contents

1	Introduction	1
2	Least squares fitting	2
2.1	Build up	2
2.2	Line of best fit	3
2.3	In class exercises pt. I	4
2.4	Hyperplane of best fit	5
2.5	Why Equation (2.4) works	5

1 Introduction

Data analysis and more broadly Data Science is a vast and important field within Computer Science and Mathematics. At the core, the goal is to make sense of data, which can be measurements, survey results, behavior patterns, etc. Often this data comes to us in a very “high dimension”. That is, there are so many variables that it is impossible to visualize, and even in low dimensions, it may not be clear what the best conclusion is based on the analysis.

A few references seem to agree that the *total data* on all computers is something like **10^{23} bytes** or about **100 zettabytes**. While all of this data is not concentrated in one organization, we still require highly sophisticated tools to make sense of a huge amount of data. My goal with this course is that you will have a solid foundation with some standard tools. From this bedrock one could explore more sophisticated methods of data analysis more easily.

We will consider four key topics:

1. Least Squares Fitting,
2. Principal Component Analysis,
3. k -Means and Hierarchical Clustering,
4. Nearest Neighbors and the Johnson–Lindenstrauss Theorem

We will be working with the assumption that **the data we care about is preserved by orthogonal and linear transformations**. This is not true with all data—for example, one should not take (proper) linear combinations of people. However, for data like grams of different kinds of food, this is completely plausible. This assumption will not always be necessary, but we will just keep this in mind.

Half of our time will be spent bringing these ideas to life and getting our hands dirty. We will be working with Jupyter Notebooks to build familiarity with the concepts we will discuss. This will be done using Python and standard data analysis packages like Pandas.

2 Least squares fitting

This method of analysis is both simple and powerful. Like with most things in mathematics, without some good guiding examples, we can get lost in the formulas.

2.1 Build up

Suppose we have the following data as seen in Figure 2.1. (Maybe a company produces parts once per month in lots that vary in size depending on demand. We write i for the production run, x_i for the lot size, and y_i for the hours of labor.)

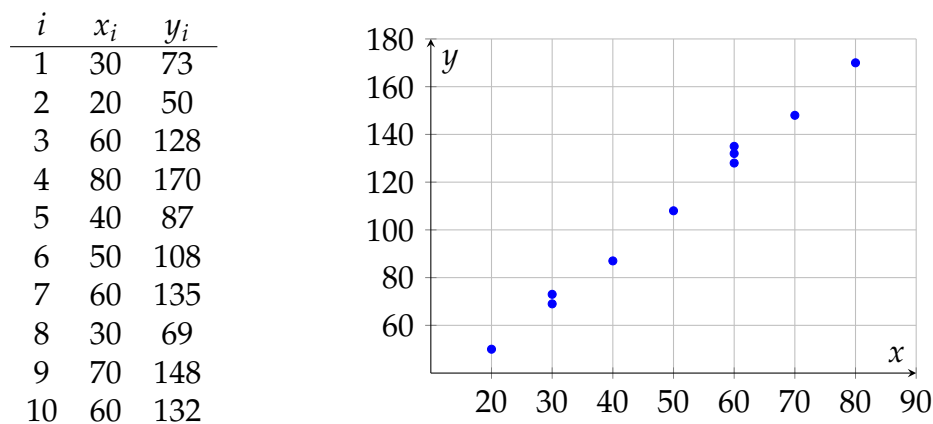


Figure 2.1: Data points exhibiting a linear relationship.

It seems clear from the plot that the data fits a geometric pattern—there is a linear phenomenon. If we try to find the line that is somehow closest to all the data points, we might draw something like in Figure 2.2.

Although the line is not a perfect fit, it seems to tell us something about the relationship between the lots size and the amount of hours.

Questions:

- What makes this line “better” than alternatives?
- How are we quantifying “better”?

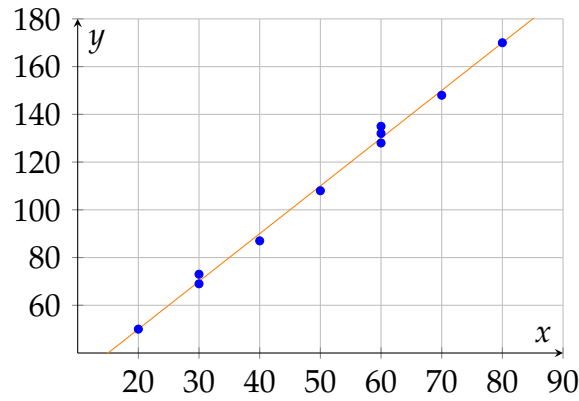


Figure 2.2: Line of best fit with data points.

- Why are we using a line?

We will answer the first question later, probably. Let us consider the question of quantifying “better”. Least squares fitting is all about minimizing the squares of differences between the line and the actual data points. We will make this precise very soon.

2.2 Line of best fit

We know that the equation of a non-vertical line has the form

$$y = b_0 + b_1x.$$

If we had n data points of the form (x_i, y_i) , we could choose b_0 and b_1 to minimize the following sum

$$S(b_0, b_1) = \sum_{i=1}^n (y_i - (b_0 + b_1x_i))^2.$$

We can even solve for these values. Since S is a function in terms of b_0 and b_1 , all possible minima occur when the partial derivatives of S are 0. In other words, the minima arise as values (b_0, b_1) such that

$$\begin{aligned} \frac{\partial S}{\partial b_0} &= -2 \sum_{i=1}^n (y_i - (b_0 + b_1x_i)) = 0, \\ \frac{\partial S}{\partial b_1} &= -2 \sum_{i=1}^n (x_i y_i - x_i(b_0 + b_1x_i)) = 0. \end{aligned}$$

These equations are linear equations, so we can solve for these with techniques from linear algebra. This mean, we need to solve two equations in the unknown b_0 and b_1 :

$$\begin{aligned} nb_0 + b_1 \sum x_i &= \sum y_i, \\ b_0 \sum x_i + b_1 \sum x_i^2 &= \sum x_i y_i. \end{aligned} \tag{2.1}$$

Using the data from our example in Section 2.1, we have

$$\sum x_i = 500, \quad \sum y_i = 1100, \quad \sum x_i^2 = 28400, \quad \sum x_i y_i = 61800.$$

Thus, the equations we need to solve are

$$\begin{aligned} 10b_0 + 500b_1 &= 1100, \\ 500b_0 + 28400b_1 &= 61800, \end{aligned}$$

which yield $b_0 = 10$ and $b_1 = 2$. Going back to the context of the initial problem: this solution tells us that by increasing the lot size by one, we expect to increase the labor hours by two.

2.2.1 Written as matrices

Let us write the equations in (2.1) with matrices. This might seem like overkill at this stage, but it will set us up nicely to generalize. Let

$$X = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}, \quad Y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad B = \begin{pmatrix} b_0 \\ b_1 \end{pmatrix}. \quad (2.2)$$

Therefore, the equations in (2.1) are equivalent to the single matrix equation:

$$X^t X B = X^t Y. \quad (2.3)$$

If $X^t X$ is invertible, then $B = (X^t X)^{-1} X^t Y$.

2.3 In class exercises pt. I

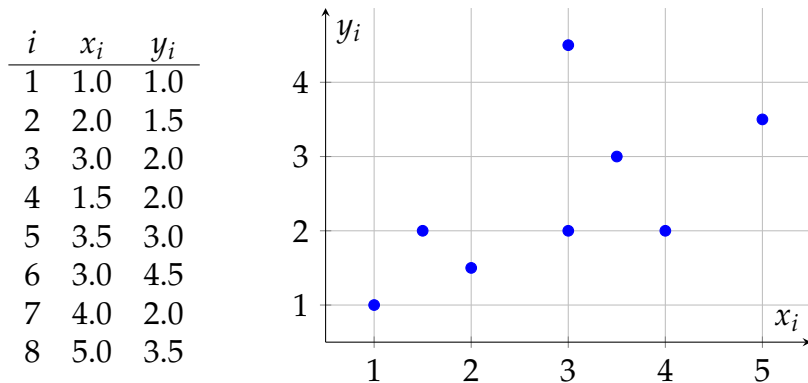
- (a) With X , Y , and B as defined in (2.2), show that

$$X^t X = \begin{pmatrix} n & \sum x_i \\ \sum x_i & \sum x_i^2 \end{pmatrix}, \quad X^t Y = \begin{pmatrix} \sum y_i \\ \sum x_i y_i \end{pmatrix}.$$

- (b) Show that $X^t X B = X^t Y$ is equivalent to (2.1):

$$\begin{aligned} nb_0 + b_1 \sum x_i &= \sum y_i, \\ b_0 \sum x_i + b_1 \sum x_i^2 &= \sum x_i y_i. \end{aligned}$$

- Find a least squares fitting line to the following data and draw in the line:



(Round b_0 and b_1 to the nearest half integer.)

2.4 Hyperplane of best fit

Now we go to the general case. Suppose we have $p - 1$ independent variables and 1 dependent variable, where $p \geq 2$. We assume we have n data points of the form

$$(x_{i1}, x_{i2}, \dots, x_{i,p-1}, y_i) \in \mathbb{R}^p.$$

The least squares fitting for these data is a hyperplane of the form

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_{p-1}x_{p-1}.$$

To solve for the values b_i , we do as we did before. We define matrices

$$X = \begin{pmatrix} 1 & x_{11} & x_{12} & \dots & x_{1,p-1} \\ 1 & x_{21} & x_{22} & \dots & x_{2,p-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{n,p-1} \end{pmatrix}, \quad Y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad B = \begin{pmatrix} b_0 \\ b_1 \\ \vdots \\ b_{p-1} \end{pmatrix}.$$

As before, the values we want are given by the equation

$$X^t X B = X^t Y. \tag{2.4}$$

2.5 Why Equation (2.4) works

The heart of least squares is (Euclidean) distance. The distance between two points $x = (x_1, \dots, x_p)$ and $y = (y_1, \dots, y_p)$ in $\mathbb{R}^p$ is

$$d(x, y) = \|x - y\| = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_p - y_p)^2}.$$

For a vector $v = (v_1, \dots, v_p) \in \mathbb{R}^p$, the **length** of v is

$$\|v\| = d(0, v) = \sqrt{v_1^2 + v_2^2 + \dots + v_p^2}.$$

Recall that the dot product of two (column) vectors u and v is

$$u \cdot v = u^t v = u_1 v_1 + u_2 v_2 + \dots + u_p v_p.$$

Thus, the length of v is $\|v\| = \sqrt{v \cdot v}$; in other words $\|v\|^2 = v \cdot v$. In addition, if $u \cdot v = 0$, we say that u and v are **orthogonal** (or perpendicular).

Definition 2.1. Let U be a subspace of a vector space V . Define $U^\perp$, called the **orthogonal complement** (or **perp-space**) of U , to be

$$U^\perp = \{v \in V \mid \forall u \in U, u \cdot v = 0\}.$$

Lemma 2.2. Let U be a subspace of $V = \mathbb{R}^p$.

1. $U^\perp$ is a subspace of V .
2. $(U^\perp)^\perp = U$.
3. $U \cap U^\perp = \{0\}$.
4. $\dim(U) + \dim(U^\perp) = p$.

Proof. Try to prove this yourself. If you need help, you can read [5, Sec. 6.2]. $\square$

The goal of least squares is to *minimize distance*; more specifically to minimize $\|Y - XB\|$. Note that the column vector Y has entries that are the *actual* y_i values, and the column vector

$$XB = \begin{pmatrix} B \cdot (1, x_{11}, x_{12}, \dots, x_{1,p-1}) \\ B \cdot (1, x_{21}, x_{22}, \dots, x_{2,p-1}) \\ \vdots \\ B \cdot (1, x_{n1}, x_{n2}, \dots, x_{n,p-1}) \end{pmatrix} = \begin{pmatrix} b_0 + b_1x_{11} + b_2x_{12} + \dots + b_{p-1}x_{1,p-1} \\ b_0 + b_1x_{21} + b_2x_{22} + \dots + b_{p-1}x_{2,p-1} \\ \vdots \\ b_0 + b_1x_{n1} + b_2x_{n2} + \dots + b_{p-1}x_{n,p-1} \end{pmatrix}$$

Therefore, $\|Y - XB\|$ is the square root of a sum of squares of the form

$$y_i - b_0 + b_1x_{i1} + b_2x_{i2} + \dots + b_{p-1}x_{i,p-1}.$$

Hence minimizing $\|Y - XB\|$ is the same as minimizing $\|Y - XB\|^2$, which is a sum of *squares*. To achieve this, we first need to discuss orthogonal decompositions.

References

- [1] Christos Sotiriou, Soek-Ying Neo, Lisa M. McShane, Edward L. Korn, Philip M. Long, Amir Jazaeri, Philippe Martiat, Steve B. Fox, Adrian L. Harris, and Edison T. Liu, *Breast cancer classification and prognosis based on gene expression profiles from a population-based study*, Proceedings of the National Academy of Sciences **100** (2003), no. 18, 10393–10398.
- [2] Peter Frankl and Hiroshi Maehara, *Some geometric applications of the beta distribution*, Ann. Inst. Statist. Math. **42** (1990), no. 3, 463–474.
- [3] Google, *What is Clustering?* (2023), date accessed: 16 Oct. 2023. <https://developers.google.com/machine-learning/clustering/overview>.
- [4] Sarel Har-Peled, Piotr Indyk, and Rajeev Motwani, *Approximate nearest neighbor: towards removing the curse of dimensionality*, Theory Comput. **8** (2012), 321–350, DOI 10.4086/toc.2012.v008a014. MR2948494
- [5] Dan Margalit and Joseph Rabinoff, *Interactive Linear Algebra*, 2019, <https://textbooks.math.gatech.edu/ila/>.
- [6] Jonathon Shlens, *A tutorial on principal component analysis*, 2014, [arXiv:1404.1100](https://arxiv.org/abs/1404.1100).
- [7] Fan Yang and Sifan Liu and Edgar Dobriban and David P. Woodruff, *How to reduce dimension with PCA and random projections?*, 2021, [arXiv:2005.00511](https://arxiv.org/abs/2005.00511).